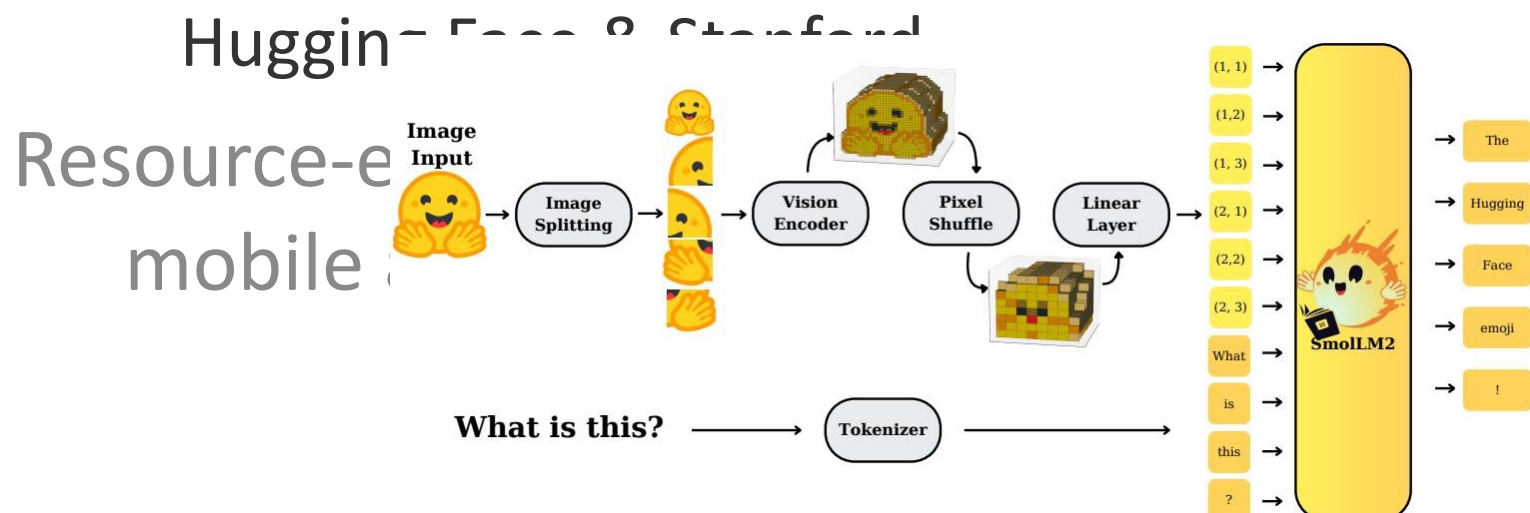


SmolVLM: Redefining Small and Efficient Multimodal Models

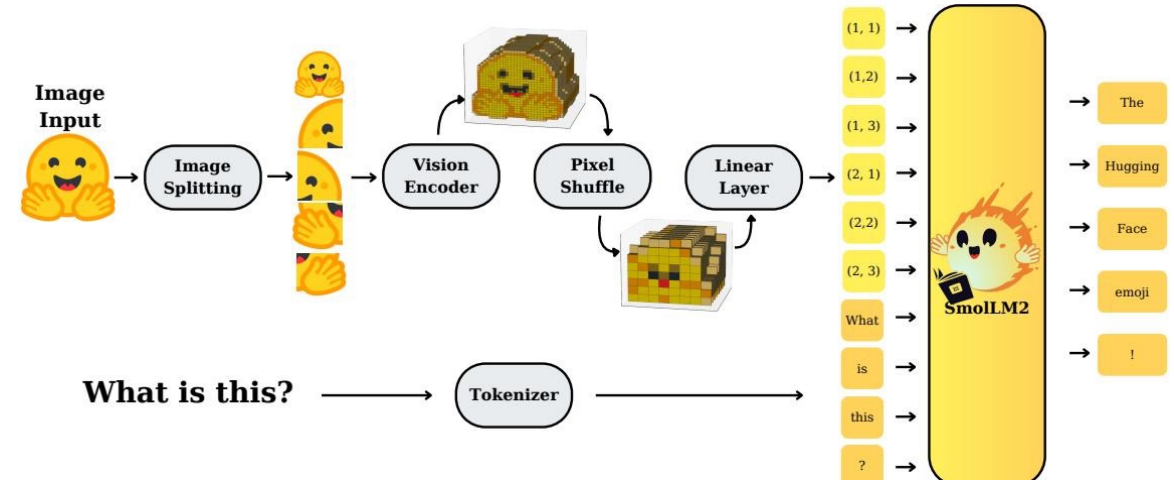


Large VLMs Are Powerful but Impractical for Edge Deployment

- State-of-the-art VLMs require massive compute and memory:
 - Typical models (e.g., Molmo-A1B-7B) require ~27.7 GB of GPU RAM.
 - Idedfics-80B uses enormous parameters making it strictly server-bound.
- Mobile and edge deployment requires sub-1GB memory footprints.
- The SmolVLM Solution:
 - Parameter count scaled down to 256M - 2.2B range.
 - Employs aggressive compression of visual tokens.
 - Retains competitive capabilities despite structural constraints.

SmolVLM Architecture: Image Splitting, Pixel Shuffle & SmolLM2

- Four-Stage Architecture Pipeline:
- 1. Image Splitting: High-res images are partitioned into smaller sub-images.
- 2. Vision Encoder: Sub-images encoded via SigLIP.
- 3. Pixel Shuffle: Space-to-depth compression ($r=4$) heavily reduces tokens.
- 4. SmolLM2 Backbone: Tokens are projected linearly and concatenated.
- Note: Video processing follows a



Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

- Finding 1: Balanced Encoder-LM Compute

Compact VLMs benefit significantly from smaller vision encoders.

A 93M SigLIP-B/16 encoder outperforms a 428M SigLIP-SO400M when paired with a 135M LM.

The larger 428M encoder only justifies its overhead at the 1.7B LM scale.

Encoder Model	Parameters	Language Model	Performance Result
SigLIP-B/16	93M	135M LM	Higher Performance
SigLIP-SO400M	428M	135M LM	Lower Performance
SigLIP-SO400M	428M	1.7B LM	Overhead Justified

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16×

- Finding 3: Aggressive Token Compression

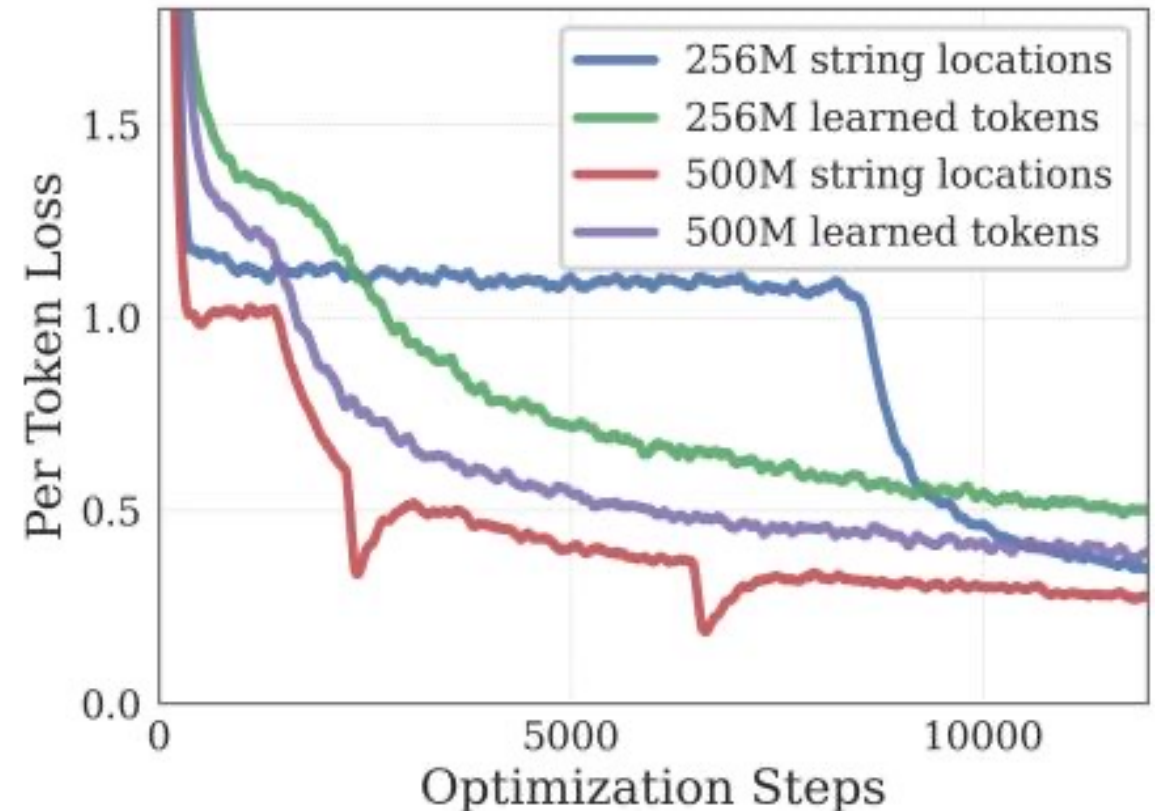
Pixel shuffle rearranges spatial features into channels.

Reduces visual tokens by a factor of r^2 .

Small VLMs benefit from $r=4$ (16× token reduction).

Eases attention overhead and improves long-context modeling.

By contrast, larger models typically prefer $r=2$.



Extending Context from 2K to 16K Tokens

- Finding 2: Extended Context Length

Extending context from 2K to 16K tokens unlocks higher resolutions.

Achieved by increasing the RoPE base from 10K to 273K.

Requires fine-tuning on mixed long/short-context data.

- Model Context Capacities:

SmolVLM-2.2B: Utilizes full 16K context window.

SmolVLM-256M & 500M: Utilize 8K context windows.

SmolVLM Achieves Competitive Performance at Sub-1GB RAM

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo-A1B-7B)
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo-A1B-7B)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo-A1B-7B)
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo-A1B-7B)
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo-A1B-7B)
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo-A1B-7B)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo-A1B-7B)
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo-A1B-7B)
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo-A1B-7B)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2-2B)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2-2B)
Average	All Benchmarks	44.0%	51.0%	59.8%	—

- RAM Usage vs Competitors:

SmolVLM 256M: 0.8 GB | SmolVLM 500M: 1.2 GB | SmolVLM 2.2B: 4.9 GB

Molmo-A1B-7B requires 27.7 GB GPU RAM.

SmolVLM-256M surpasses the 300× larger Idefics-80B despite size.

Real-Time Inference on iPhones & Apple Silicon

- HuggingSnap iOS App:

Runs completely on-device, processing photos and videos locally.

No cloud dependency ensures privacy and speed.

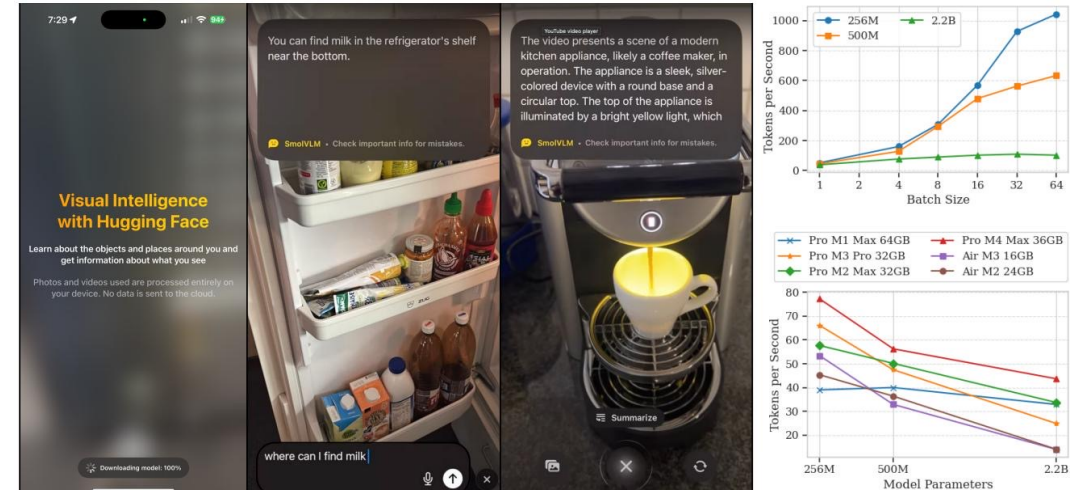
- Throughput Benchmarks:

SmolVLM-256M achieves ~80 tokens/sec on Pro M1 Max 64GB.

Reaches nearly 1000 tokens/sec at batch size 64 on high-end devices.

MacBook Air M3 16GB yields ~55 tokens/sec for the 256M model.

Throughput scales roughly linearly with batch size



Key Takeaways: Design Principles for Efficient VLMs

- 1. Sub-1GB inference is practically achievable: Models like SmolVLM-256M use just 0.8 GB RAM.
- 2. Balance encoder-LM compute: Smaller encoders paired with small LMs outperform larger encoders.
- 3. Extend context length: Scaling from 2K to 16K with RoPE adjustments significantly boosts capabilities.
- 4. Use aggressive token compression: Pixel shuffle ($r=4$) achieves 16× token reduction.
- 5. Fully open-source release: Data, code, and HuggingSnap app freely available for on-device AI research.